

Hotspot Karhutla - Panduan

OLAP Hotspot - Sistem Analitik Titik Panas Kebakaran Indonesia

Sistem data warehouse dan OLAP untuk analisis titik panas (hotspot) kebakaran hutan dan lahan di Indonesia. Data deteksi kebakaran diambil dari NASA FIRMS, diperkaya dengan data cuaca dan lokasi administratif, disimpan pada ClickHouse, disajikan melalui REST API, lalu divisualisasikan pada dashboard peta.

Dokumen ini menjelaskan cara menjalankan sistem secara lokal dan cara melakukan deployment ke server produksi.

Arsitektur

Alur data secara ringkas:

```
NASA FIRMS -> Airflow (ETL) -> ClickHouse (galaxy schema) -> REST API (Go, 3 replika) -> nginx (reverse proxy + load balancer) -> Dashboard (Astro)
```

Komponen sistem:

Komponen	Direktori	Teknologi
REST API	<code>api/</code>	Go 1.24, chi, ClickHouse, Redis, Swagger
Pipeline ETL	<code>de/</code>	Python 3.11, Polars, Apache Airflow, DuckDB
Dashboard	<code>app/</code>	Astro (build statis), React, Tailwind, Bun
Reverse proxy	<code>nginx/</code>	nginx

Basis data pendukung: ClickHouse (data warehouse OLAP), Redis (cache API), PostgreSQL (metadata Airflow).

Model data warehouse menggunakan galaxy schema (fact constellation): terdapat dua tabel fakta, yaitu `fact_hotspot` dan `fact_weather`, yang berbagi dimensi bersama (`dim_location`, `dim_period`, `dim_satellite`, `dim_confidence`, `dim_weather_condition`).

Prasyarat

- Docker dan Docker Compose v2 untuk menjalankan seluruh stack.
- Opsional, untuk pengembangan per-komponen: Go 1.24+, Bun 1.3+, serta Python 3.11+ (disarankan melalui `uv`).

- Kunci API NASA FIRMS (gratis) untuk menjalankan ETL. Dapat diperoleh di <https://firms.modaps.eosdis.nasa.gov/api/>.

Struktur Direktori

```
.
├── api/                                # REST API (Go)
│   ├── domain/                        # entity, DTO, envelope response, sentinel error
│   ├── service/                       # logika bisnis + interface repository
│   ├── internal/
│   │   ├── repository/                # implementasi ClickHouse
│   │   └── rest/                     # handler HTTP + anotasi Swagger
│   ├── database/                     # koneksi ClickHouse dan Redis
│   ├── docs/                         # Swagger (hasil generate)
│   └── main.go
├── de/                                # Pipeline ETL (Python)
│   ├── src/etl/                      # extractor, transformer, loader, clients
│   ├── infra/                       # Dockerfile Airflow, init.sql ClickHouse/Postgres
│   ├── data/                        # geocoder.duckdb (basis data geocoder offline)
│   └── pyproject.toml
├── app/                              # Dashboard Astro + Dockerfile
├── nginx/                            # default.conf reverse proxy + load balancer
├── .env.example                      # contoh konfigurasi environment
└── docker-compose.yml               # orkestrasi seluruh stack
```

Konfigurasi

Seluruh konfigurasi dipusatkan pada satu berkas `.env` di direktori root. Salin berkas contoh, kemudian isi nilainya.

```
cp .env.example .env
```

Nilai minimum yang wajib diisi:

- `POSTGRES_PASSWORD` - kata sandi basis data metadata Airflow. Nilai ini harus konsisten dengan kata sandi pada `AIRFLOW__DATABASE__SQL_ALCHEMY_CONN`.
- `AIRFLOW__CORE__FERNET_KEY` - dibuat dengan perintah berikut:

```
python -c "from cryptography.fernet import Fernet;
          print(Fernet.generate_key().decode())"
```

- `AIRFLOW__WEBSERVER__SECRET_KEY` - string acak, misalnya `openssl rand -hex 32`.
- `NASA_FIRMS_API_KEY` - kunci API NASA FIRMS untuk proses ETL.

Konfigurasi geocoder (sudah disetel benar pada `.env.example`):

- `USE_OFFLINE_GEOCODER=true` - menggunakan geocoder offline berbasis DuckDB, bukan API BMKG.

- `GEOCODER_DB_PATH=/opt/airflow/data/geocoder.duckdb` - lokasi basis data geocoder di dalam kontainer.

Berkas `de/data/geocoder.duckdb` sudah disertakan dan otomatis ter-mount ke kontainer Airflow. Bila berkas tersebut tidak ada, pipeline akan mengunduhnya sekali dari GitHub release saat pertama dijalankan.

Menjalankan Secara Lokal

1. Mengunduh data seed

Data seed ClickHouse (hasil ekspor dari produksi) tidak disertakan di dalam repositori agar repositori tetap ringan. Berkas seed di-host sebagai aset GitHub Release dan diunduh sekali sebelum menjalankan stack:

```
./scripts/fetch-seed.sh
```

Skrip ini mengunduh berkas Native ter-kompres ke `de/infra/clickhouse/seed/`. Langkah ini opsional: bila dilewati, stack tetap berjalan namun ClickHouse mulai tanpa data (dapat diisi melalui ETL).

2. Menyalakan seluruh stack

```
docker compose up -d --build
```

Perintah ini menyalakan PostgreSQL, Redis, ClickHouse, Airflow (webserver, scheduler, init), tiga replika API, frontend, dan nginx. Skema ClickHouse (tabel `dim_*`, `fact_*`, `staging_*`) dibuat otomatis dari `de/infra/clickhouse/init.sql` saat ClickHouse pertama kali dijalankan.

Pada eksekusi pertama (volume masih kosong), bila berkas seed sudah diunduh pada langkah 1, ClickHouse otomatis memuatnya melalui skrip `de/infra/clickhouse/load-seed.sh` setelah skema terbentuk. Dengan demikian dashboard langsung berisi data nyata tanpa perlu menjalankan ETL terlebih dahulu. Bila direktori seed kosong, langkah ini dilewati dan data dapat diisi melalui ETL.

Setelah seluruh layanan berstatus sehat, akses melalui nginx edge:

- Dashboard: `http://localhost:8090`
- API (melalui nginx): `http://localhost:8090/api/v1/hotspots/filter-options`
- Health check: `http://localhost:8090/health`
- Airflow: `http://localhost:8080` (pengguna dan kata sandi default `admin / admin`, diatur pada `.env`)

3. Menjalankan ETL untuk mengisi data

Pada awalnya ClickHouse hanya berisi skema tanpa data. Isi data melalui DAG Airflow.

```
# Mengaktifkan DAG harian
docker compose exec airflow-scheduler airflow dags unpause hotspot_daily

# Memicu satu kali eksekusi manual
docker compose exec airflow-scheduler airflow dags trigger hotspot_daily
```

Atau lakukan melalui antarmuka Airflow di <http://localhost:8080>. DAG `hotspot_daily` mengambil data hari berjalan dari NASA FIRMS, melakukan reverse-geocoding secara offline, mengambil data cuaca, lalu memuat ke ClickHouse. Tersedia pula DAG `hotspot_backfill` untuk memproses data historis per bulan.

Catatan: ketersediaan data harian bergantung pada waktu lintasan satelit. Bila dijalankan pada dini hari UTC, data hari tersebut mungkin belum dipublikasikan oleh NASA sehingga hasilnya masih kosong.

Verifikasi

1. Status kontainer

```
docker compose ps
```

Seluruh layanan inti (`clickhouse`, `redis`, `postgres`, `api-1/2/3`) diharapkan berstatus `healthy`.

2. Kesehatan API dan skema

```
curl http://localhost:8090/health
# {"status":"ok","clickhouse":"connected"}

curl "http://localhost:8123/?query=SHOW+TABLES+FROM+hotspot" -u 'default:'
# dim_confidence, dim_location, dim_satellite, fact_hotspot, ...
```

3. Endpoint API

Seluruh endpoint berada di bawah prefiks `/api/v1/hotspots`.

Endpoint	Fungsi
GET /	Daftar deteksi hotspot (paginasi)
GET /geojson	Hotspot dalam format GeoJSON untuk peta
GET /summary	Ringkasan dashboard (wilayah teratas, distribusi, statistik)
GET /filter-options	Opsi filter (tingkat keyakinan, satelit)
GET /periods	Opsi periode drill-down (tahun, semester, kuartal, bulan, minggu)
GET /locations	Hierarki lokasi beserta jumlah hotspot

Contoh:

```
curl http://localhost:8090/api/v1/hotspots/summary
curl "http://localhost:8090/api/v1/hotspots/gejson?year=2026"
```

4. Dashboard

Buka `http://localhost:8090` pada peramban. Setelah ETL mengisi data, peta akan menampilkan titik-titik hotspot beserta chart dan filter.

5. Pengujian API

```
cd api
go test ./... -count=1 -cover
```

Pengembangan Per-Komponen

API (Go):

```
cd api
go build ./...
go test ./...
swag init -g main.go -o docs --parseDependency --parseInternal # regenerasi
                        Swagger
mockery                  # regenerasi
                        mock (baca .mockery.yml)
```

Untuk mengaktifkan Swagger UI, jalankan API dengan `ENABLE_SWAGGER=true`, lalu buka `/swagger/index.html`.

Frontend (Astro):

```
cd app
bun install
bun run dev          # mode pengembangan di http://localhost:4321
bun run build        # build statis ke direktori dist/
```

Nilai `PUBLIC_API_URL` di-bake saat build dan harus menunjuk ke alamat API (pada setup lokal: `http://localhost:8090`).

Deployment ke Produksi

Pada produksi, aplikasi ditempatkan di server (contoh direktori: `/opt/olap-hotspot`). nginx berjalan langsung di host (bukan sebagai kontainer) dan bertindak sebagai reverse proxy sekaligus load balancer ke tiga replika API.

1. Menyalin kode ke server

```
rsync -az --delete \  
  --exclude='.git' --exclude='node_modules' --exclude='dist' \  
  --exclude='.astro' --exclude='__pycache__' \  
./ root@SERVER:/opt/olap-hotspot/
```

Pastikan berkas `.env` produksi sudah ada di server dan tidak ditimpa.

2. Membangun ulang dan menerapkan API tanpa waktu henti (rolling restart)

Karena API berjalan dalam tiga replika, penerapan dilakukan satu per satu agar dua replika lain tetap melayani.

```
cd /opt/olap-hotspot  
  
# Membangun ulang image API  
docker compose build api-1 api-2 api-3  
  
# Menerapkan satu per satu, verifikasi kesehatan di antara langkah  
docker compose up -d --no-deps --force-recreate api-1  
curl -f http://127.0.0.1:3001/health  
  
docker compose up -d --no-deps --force-recreate api-2  
curl -f http://127.0.0.1:3002/health  
  
docker compose up -d --no-deps --force-recreate api-3  
curl -f http://127.0.0.1:3003/health
```

Opsi `--no-deps` memastikan ClickHouse, Redis, dan Airflow tidak ikut di-restart.

3. Menerapkan perubahan ETL

```
# Setelah kode di de/src disinkronkan ke server  
docker compose up -d --no-deps --force-recreate airflow-scheduler airflow-  
  webserver
```

4. Rollback

Sebelum penerapan, disarankan mencadangkan kode dan menandai image lama.

```
# Cadangkan kode dan tandai image saat ini  
tar czf api.bak.${date +%Y%m%d-%H%M%S}.tar.gz api  
docker tag olap-hotspot-api-1 olap-hotspot-api-1:rollback  
  
# Bila perlu kembali, pulihkan kode lama lalu terapkan ulang image rollback  
docker tag olap-hotspot-api-1:rollback olap-hotspot-api-1:latest  
docker compose up -d --no-deps --force-recreate api-1 api-2 api-3
```

5. nginx pada host (produksi)

Contoh konfigurasi reverse proxy dan load balancer terdapat pada `nginx/default.conf`. Pada host produksi, arahkan upstream ke port replika API (`127.0.0.1:3001`, `3002`, `3003`) dan sajikan hasil build statis frontend.

Daftar Port

Layanan	Port host
nginx edge (dashboard + API)	8090
API replika 1/2/3 (akses langsung)	3001 / 3002 / 3003
ClickHouse (HTTP / native)	8123 / 9000
Redis	6379
PostgreSQL	5432
Airflow web	8080

Menghentikan

```
docker compose down          # menghentikan seluruh kontainer
docker compose down -v       # sekaligus menghapus volume (data ClickHouse,
                             Postgres, Redis)
```

Catatan

- ClickHouse dimulai dalam keadaan kosong (hanya skema). Data deteksi diisi oleh ETL yang membutuhkan `NASA_FIRMS_API_KEY`.
- Response API menggunakan amplop yang konsisten: `{message, success, data}`.
- Reverse-geocoding menggunakan basis data DuckDB offline (`geocoder.duckdb`) sehingga tidak bergantung pada ketersediaan API BMKG.
- Sebagian endpoint menyetel header `Cache-Control` untuk mendukung caching pada CDN atau edge.